Reporte

Nombre: Blanca Jocelin Vazquez Reyna **Nombre**: Ruben Antonio Herrera Rodriguez **Nombre**: Aaron Sebastian Torres Cabriales	**Matrícula**: 2152983 **Matrícula**: 2149174 **Matrícula**:2153261
Nombre del curso: Componentes de lógica de negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Reporte Unidad 4
Fecha: 26/10/2026	
Bibliografia (Formato APA)	

Objetivo: Definir, documentar y validar mediante pruebas automatizadas y un sistema basado en Domain-Driven Design (DDD) las reglas de negocio del sistema TechStore, abarcando el ciclo de vida de los pedidos, la validación de clientes y límites de artículos, la aplicación de reglas de descuento mediante el patrón de especificaciones (*Specifications*), y el control de transiciones de estado con su respectivo registro de eventos de dominio (*Domain Events*). El análisis busca asegurar el cumplimiento de las restricciones operativas y demostrar el correcto funcionamiento de la arquitectura mediante pruebas unitarias (pytest) y pruebas de integración de la API REST.

Reporte

Datos:

Clientes

cliente_id	nombre	es_vip	activo
C001	Ana Torres	1	1
C002	Luis Medina	0	1
C003	Carla Ruiz	1	0
C004	Diego Peña	0	0
C005	Sofia Lara	1	1
C006	Pedro Soto	0	1
C007	Mariana Gil	1	1
C008	Raul Vega	0	1
C009	Elena Mora	1	1
C010	Jorge Luna	0	1
C011	Valeria Rios	1	1
C012	Oscar Nieto	0	1

Reporte

Pedidos

pedido_id	sku	cantidad
O001	SKU001	2
O001	SKU002	2
O002	SKU001	1
O002	SKU002	1
O003	SKU001	2
O003	SKU002	2
O004	SKU004	1
O005	SKU001	2
O005	SKU002	2
O006	SKU008	1
O008	SKU003	20
O009	SKU003	19
O010	SKU003	1
O010	SKU009	1
O011	SKU001	1

Reporte

pedido_id	sku	cantidad
O011	SKU002	2
O012	SKU004	1
O013	SKU001	1
O014	SKU001	1
O015	SKU002	1
O016	SKU011	1

Escenarios

caso_id	pedido_id	accion	objetivo	pista
E01	O001	evaluar_descuento	10%	AND VIP + monto >500 + unidades >=4 + NOT promo
E02	O002	evaluar_descuento	0%	No VIP, total menor a 5000
E03	O003	confirmar	rechazo	Cliente inactivo
E04	O005	evaluar_descuento	0%	Promocion activa bloquea descuento
E05	O006	evaluar_descuento	5%	OR total >5000 y sin promocion

Reporte

caso_id	pedido_id	accion	objetivo	pista
E06	O007	confirmar	rechazo	Pedido sin articulos
E07	O008	confirmar	exito	20 unidades permitidas
E08	O009	agregar SKU003 x2	rechazo	21 unidades excede limite
E09	O010	evaluar_descuento	0%	Umbral estricto >500
E10	O011	evaluar_descuento	0%	Solo 3 unidades, requiere 4
E11	O012	confirmar y enviar sin guia	rechazo envio	La confirmacion es valida pero falta guia
E12	O013	confirmar dos veces	rechazo segunda	No permite confirmar desde CONFIRMED
E13	O014	confirmar, enviar y cancelar	rechazo cancelacion	No permite cancelar desde SHIPPED
E14	O015	confirmar, enviar y devolver	exito	Termina en RETURNED
E15	O016	confirmar, enviar y entregar	exito	Termina en DELIVERED

Reporte

Pedidos

pedido_id	cliente_id	tiene_promocion	estado_inicial	descripcion
O001	C001	0	DRAFT	VIP con 4 unidades: descuento 10%
O002	C002	0	DRAFT	No VIP: descuento 0%
O003	C003	0	DRAFT	VIP inactivo: no confirmar
O004	C004	0	DRAFT	Cliente normal inactivo
O005	C005	1	DRAFT	Promocion activa excluye descuentos
O006	C006	0	DRAFT	Total superior a 5000: descuento 5%
O007	C007	0	DRAFT	Pedido vacio: no confirmar
O008	C008	0	DRAFT	20 unidades exactas permitidas
O009	C009	0	DRAFT	Al agregar se superan 20 unidades

Reporte

pedido_id	cliente_id	tiene_promocion	estado_inicial	descripcion
O010	C010	0	DRAFT	Total exactamente 500
O011	C011	0	DRAFT	VIP con solo 3 unidades
O012	C012	0	DRAFT	Enviar sin guia despues de confirmar
O013	C001	0	DRAFT	Confirmar dos veces
O014	C002	0	DRAFT	Cancelar despues de enviar
O015	C005	0	DRAFT	Devolucion desde SHIPPED
O016	C007	0	DRAFT	Entrega desde SHIPPED

Reporte

Productos

sku	nombre	precio_mxn	existencia
SKU001	Mouse inalambrico	600.00	35
SKU002	Teclado mecanico	450.00	20
SKU003	Cable USB-C	120.00	70
SKU004	Audifonos	850.00	14
SKU005	Monitor 24 pulgadas	3200.00	10
SKU006	SSD 1TB	1750.00	18
SKU007	Webcam HD	950.00	22
SKU008	Laptop empresarial	18000.00	7
SKU009	Hub USB	380.00	24
SKU010	Base para laptop	290.00	40
SKU011	Router Wi-Fi	2100.00	11
SKU012	Memoria USB 64GB	160.00	50

Captura de carga exitosa de los 16 pedidos:

```
(.venv) PS C:\Users\prixe\Downloads\Unidad 4> python cargar_csv.py
O001 DRAFT 4 2100.00
O002 DRAFT 2 1050.00
O003 DRAFT 4 2100.00
O004 DRAFT 1 850.00
O005 DRAFT 4 2100.00
O006 DRAFT 1 18000.00
O007 DRAFT 0 0
O008 DRAFT 20 2400.00
O009 DRAFT 19 2280.00
O010 DRAFT 2 500.00
O011 DRAFT 3 1500.00
O012 DRAFT 1 850.00
O013 DRAFT 1 600.00
O014 DRAFT 1 600.00
O015 DRAFT 1 450.00
O016 DRAFT 1 2100.00
```

Reporte

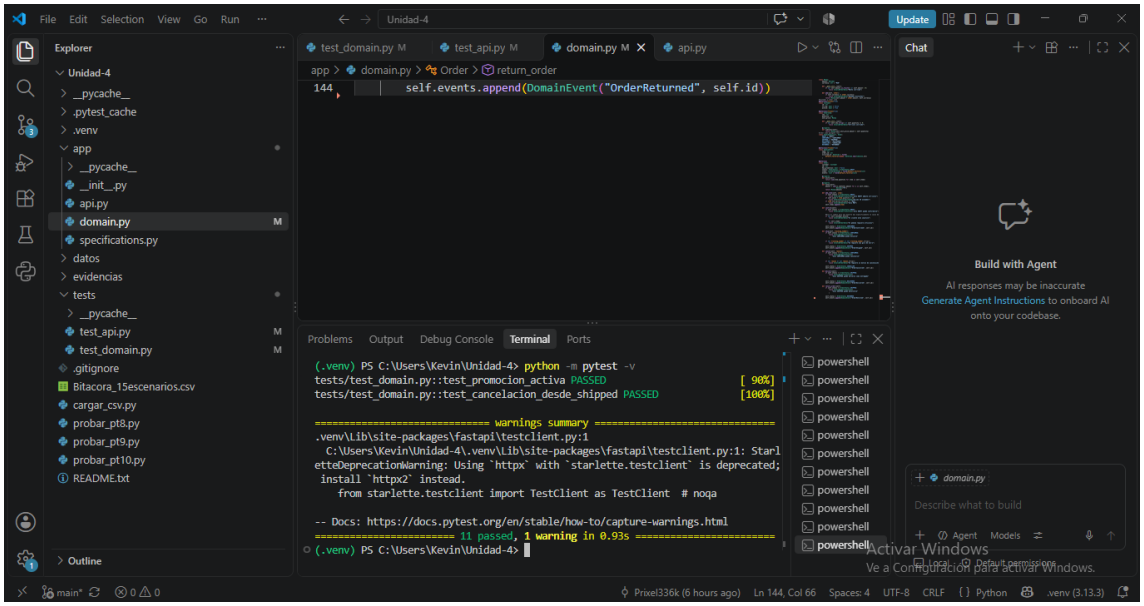
Bitácora resuelta de los 15 escenarios E01-E15:

Caso ID	Estado Inicial	Operación	Resultado Real	Explicación
E01	DRAFT	Evaluar descuento	10.00%	Regla VIP, monto mayor a 500, al menos 4 unidades y sin promoción. Esperado: 10%. La evaluación no cambia el pedido ni genera eventos.
E02	DRAFT	Evaluar descuento	0.00%	Cliente no VIP y total menor a 5000, según el escenario. Esperado: 0%. La evaluación no cambia el pedido ni genera eventos.
E03	DRAFT	Confirmar	Confirmar: Rechazo: El cliente esta inactivo. Eventos nuevos: ninguno.	Se espera rechazo por cliente inactivo. Estado final: DRAFT.
E04	DRAFT	Evaluar descuento	0.00%	La promocion activa impide aplicar ambos descuentos. Esperado: 0%. La evaluación no cambia el pedido ni genera eventos.
E05	DRAFT	Evaluar descuento	5.00%	Total mayor a 5000 y sin promocion. Esperado: 5%. La evaluación no cambia el pedido ni genera eventos.
E06	DRAFT	Confirmar	Confirmar: Rechazo: El pedido requiere articulos. Eventos nuevos: ninguno.	Se espera rechazo porque el pedido no tiene articulos. Estado final: DRAFT.
E07	DRAFT	Confirmar	Confirmar: Éxito; estado actual: CONFIRMED. Eventos nuevos: OrderConfirmed.	Unidades iniciales: 20. El limite permitido es 20. Estado final: CONFIRMED.
E08	DRAFT	Agregar SKU003 x2	Agregar SKU003 x2: Rechazo: Máximo 20 unidades. Eventos nuevos: ninguno.	Unidades antes: 19. Se intenta agregar 2 unidades. Unidades despues: 19. Se espera rechazo si se exceden 20.
E09	DRAFT	Evaluar descuento	0.00%	Un total exactamente igual a 500 no supera el umbral. Esperado: 0%. La evaluación no cambia el pedido ni genera eventos.

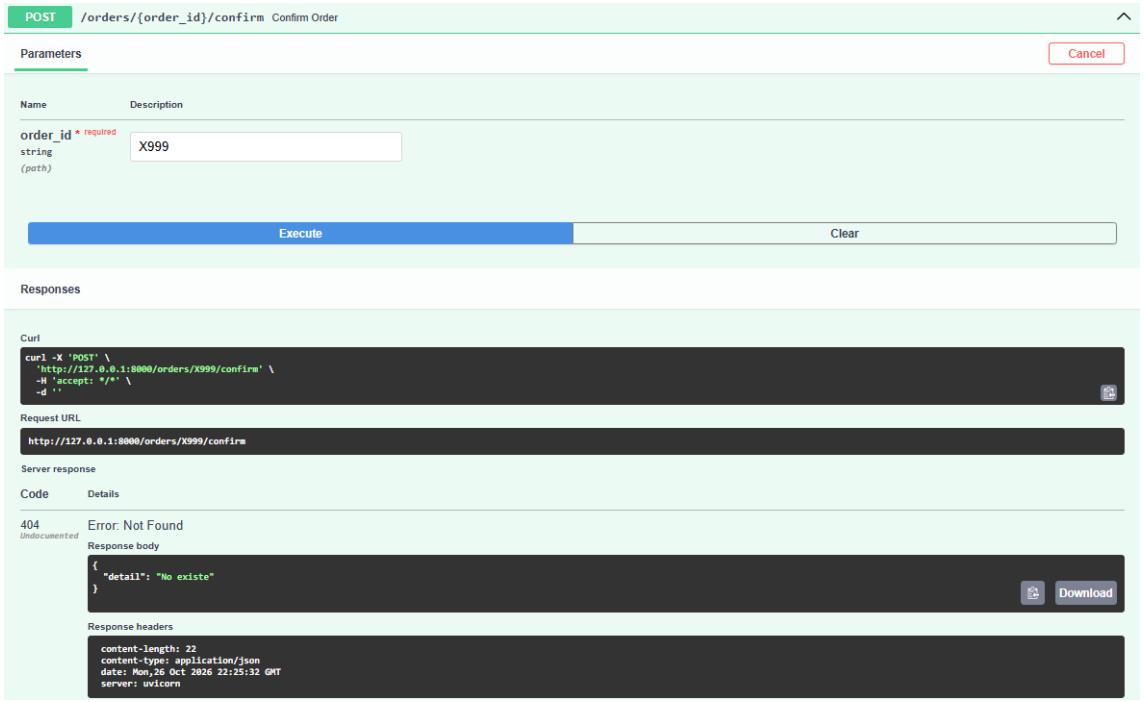
E10	DRAFT	Evaluar descuento	0.00%	La regla del 10% exige al menos 4 unidades. Esperado: 0%. La evaluación no cambia el pedido ni genera eventos.
E11	DRAFT	Confirmar y enviar sin guia	Confirmar: Éxito; estado actual: CONFIRMED. Eventos nuevos: OrderConfirmed. | Enviar sin guia: Rechazo: Se requiere una guía de envío. Eventos nuevos: ninguno.	La confirmación debe ser valida y el envio rechazado por falta de guia. Estado final: CONFIRMED.
E12	DRAFT	Confirmar dos veces	Primera confirmacion: Éxito; estado actual: CONFIRMED. Eventos nuevos: OrderConfirmed. | Segunda confirmacion: Rechazo: Solo DRAFT puede confirmarse. Eventos nuevos: ninguno.	La segunda confirmacion debe rechazarse. Estado final: CONFIRMED.
E13	DRAFT	Confirmar, enviar y cancelar	Confirmar: Éxito; estado actual: CONFIRMED. Eventos nuevos: OrderConfirmed. | Enviar: Éxito; estado actual: SHIPPED. Eventos nuevos: OrderShipped. | Cancelar: Rechazo: Solo CONFIRMED puede cancelarse. Eventos nuevos: ninguno.	La cancelacion debe rechazarse en SHIPPED. Estado final: SHIPPED.
E14	DRAFT	Confirmar, enviar y devolver	Confirmar: Éxito; estado actual: CONFIRMED. Eventos nuevos: OrderConfirmed. | Enviar: Éxito; estado actual: SHIPPED. Eventos nuevos: OrderShipped. | Devolver: Éxito; estado actual: RETURNED. Eventos nuevos: OrderReturned.	Se espera terminar en RETURNED. Estado final: RETURNED.
E15	DRAFT	Confirmar, enviar y entregar	Confirmar: Éxito; estado actual: CONFIRMED. Eventos nuevos: OrderConfirmed. | Enviar: Éxito; estado actual: SHIPPED. Eventos nuevos: OrderShipped. | Entregar: Éxito; estado actual: DELIVERED. Eventos nuevos: OrderDelivered.	Se espera terminar en DELIVERED. Estado final: DELIVERED.

Reporte

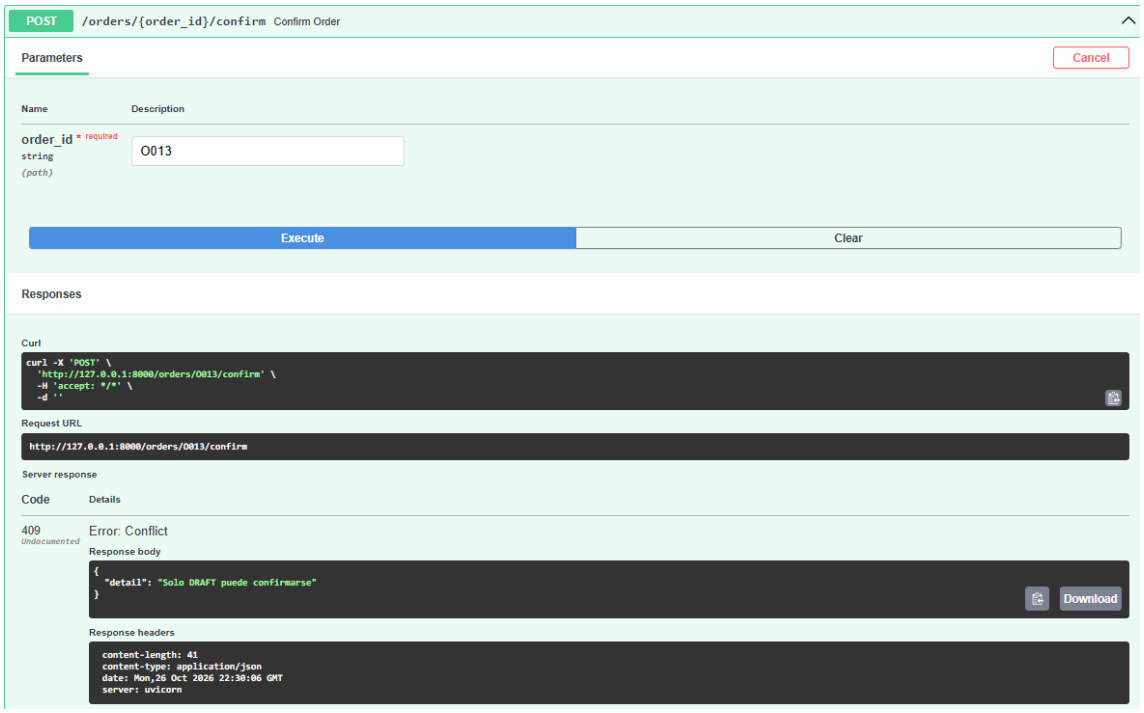
Pruebas pytest y captura de los resultados

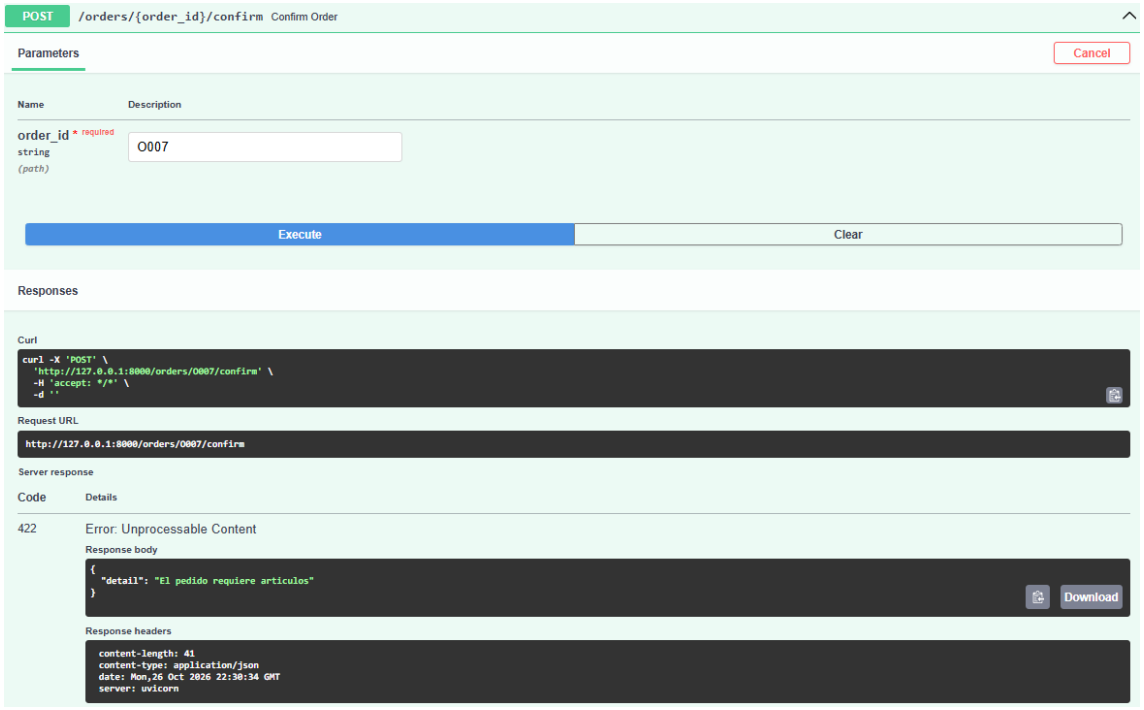


Captura de Swagger / docs mostrando una operación exitosa y una rechazada



Reporte


POST /orders/{order_id}/confirm Confirm Order
Parameters
Cancel
Name
Description
order_id * required
string
(path)
O013
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/orders/O013/confirm' \
-H 'accept: */*' \
-d ''
Request URL
http://127.0.0.1:8000/orders/O013/confirm
Server response
Code
Details
409
Undocumented
Error: Conflict
Response body
{
"detail": "Solo DRAFT puede confirmarse"
}
Download
Response headers
content-length: 41
content-type: application/json
date: Mon,26 Oct 2026 22:30:06 GMT
server: uvicorn


POST /orders/{order_id}/confirm Confirm Order
Parameters
Cancel
Name
Description
order_id * required
string
(path)
O007
Execute
Clear
Responses
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/orders/O007/confirm' \
-H 'accept: */*' \
-d ''
Request URL
http://127.0.0.1:8000/orders/O007/confirm
Server response
Code
Details
422
Error: Unprocessable Content
Response body
{
"detail": "El pedido requiere articulos"
}
Download
Response headers
content-length: 41
content-type: application/json
date: Mon,26 Oct 2026 22:30:34 GMT
server: uvicorn

Reporte

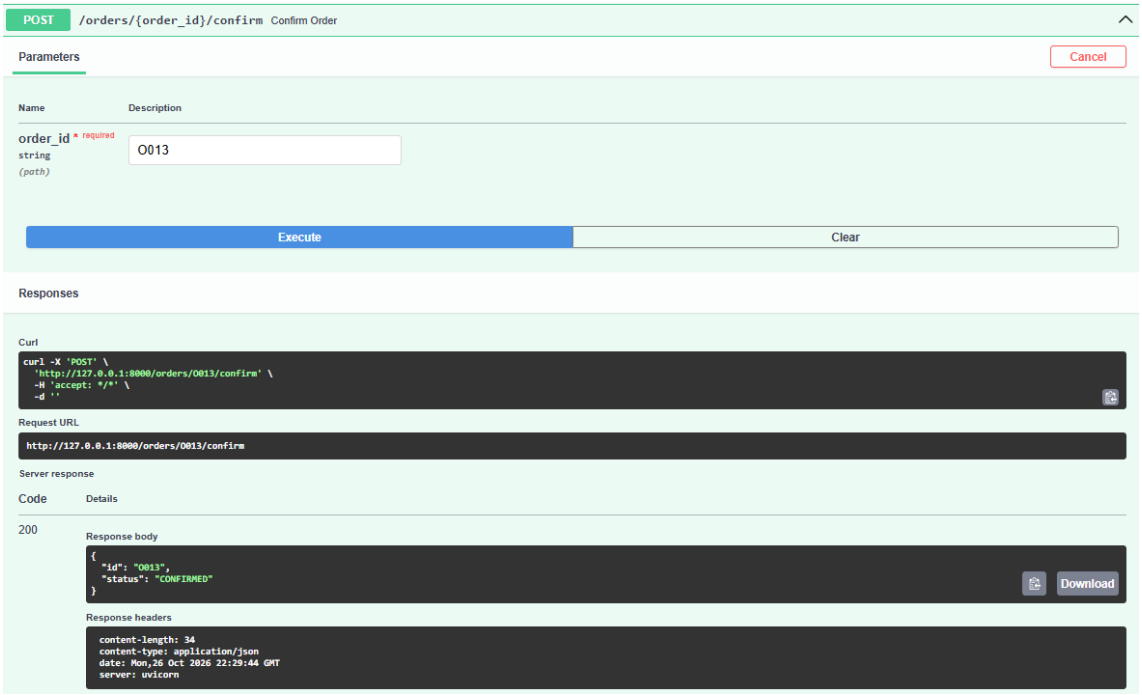


Diagrama de estados

confirm ship deliver

DRAFT ----------> CONFIRMED ----------> SHIPPED ----------> DELIVERED

| |

cancel return

| |

v v

CANCELLED RETURNED

Definiciones y Reglas de Transición

Cada operación valida el estado actual, ejecuta las reglas adicionales, cambia el estado y registra el evento correspondiente:

- confirm()
    - Estado requerido: DRAFT
    - Validación adicional: Cliente activo y pedido con artículos.
    - Evento generado: OrderConfirmed
- ship(tracking_number)
    - Estado requerido: CONFIRMED
    - Validación adicional: Guía de envío no vacía (tracking_number).
    - Evento generado: OrderShipped
- cancel (reason)
    - Estado requerido: CONFIRMED
    - Validación adicional: Motivo no vacío (reason).
    - Evento generado: OrderCancelled
- deliver()
    - Estado requerido: SHIPPED
    - Validación adicional: Ninguna.
    - Evento generado: OrderDelivered
- return_order()
    - Estado requerido: SHIPPED
    - Validación adicional: Ninguna.
    - Evento generado: OrderReturned

Nota de comportamiento: Si alguna operación no cumple con las validaciones requeridas, el pedido no cambia de estado y no se agrega ningún evento al dominio.

Explicación de los Domain Events:

En nuestro proyecto, los *Domain Events* funcionan como registros históricos inmutables que indican que un cambio importante ha ocurrido dentro de la lógica de negocio del pedido. En lugar de simplemente cambiar el estado de la orden y perder el rastro de lo que sucedió, el sistema captura estos momentos clave guardándolos en una lista interna llamada events dentro de la entidad Order.

Cada evento almacena el nombre de lo que ocurrió, el ID del pedido afectado y la fecha u hora exacta (occurred_at) en que se llevó a cabo.

Nuestro sistema dispara 5 eventos de dominio principales que van de la mano con las transiciones de estado válidas:

1. **OrderConfirmed**: Se dispara cuando un pedido que estaba en DRAFT cumple con todas las reglas de negocio (como tener un cliente activo y contener al menos un artículo), pasando exitosamente al estado CONFIRMED.
2. **OrderShipped**: Se registra cuando a un pedido confirmado se le asigna una guía de rastreo y sale a reparto, cambiando al estado SHIPPED.
3. **OrderCancelled**: Ocurre si un pedido que ya estaba CONFIRMED se cancela antes de ser enviado. Requiere obligatoriamente un motivo para poder registrarse.
4. **OrderDelivered**: Se dispara para marcar el éxito total del flujo, cuando el pedido pasa de SHIPPED a DELIVERED, indicando que ya llegó con el cliente.
5. **OrderReturned**: Se registra en casos excepcionales donde un pedido que ya había sido enviado (SHIPPED) tiene que regresar a la sucursal, pasando al estado RETURNED.

Regla clave del diseño: Los eventos se registran estrictamente **al final de una transición válida**. Si una operación falla o no cumple con las reglas, el pedido no cambia de estado y no se agrega ningún evento a la lista. Esto garantiza una auditoría completamente limpia y confiable de todo el ciclo de vida del pedido.

Preguntas para el reporte

¿Por qué se usa Decimal en lugar de float para precios? Se usa Decimal porque maneja el dinero de forma exacta, respetando los centavos y evitando los errores de redondeo o imprecisiones matemáticas que ocurren por naturaleza al usar números flotantes (float) en la programación.

¿Cuál es la diferencia entre contar renglones y sumar unidades? Contar renglones (o ítems) solo te dice cuántos tipos de productos diferentes agregaste a la lista del pedido, mientras que sumar unidades toma en cuenta la cantidad exacta de piezas de cada producto, lo cual es vital para revisar límites como el tope máximo de 20 artículos por pedido.

¿Qué ventaja tiene encapsular una regla en Specification? Permite separar las reglas de negocio complejas (como los cálculos de descuentos o validaciones especiales) del código principal de las entidades. Así, el código queda más limpio, es más fácil de mantener y puedes reutilizar o modificar esas reglas sin afectar la estructura central del dominio.

¿**Por qué los eventos se registran solo después de una transición válida?** Porque un evento de dominio (*Domain Event*) representa un hecho histórico que sí ocurrió de verdad en el sistema. Si se registrara antes de validar o si la operación fallara, estaríamos guardando información falsa sobre lo que pasó con el pedido.

¿Qué parte del código podrías reutilizar si reemplazas FastAPI por otra interfaz? Toda la capa de la lógica de negocio y las reglas del dominio (es decir, los archivos dentro de la carpeta app/domain.py y app/specifications.py). Como están completamente separadas de la parte web, podrías cambiar FastAPI por una interfaz de consola, una aplicación de escritorio o cualquier otro framework sin tener que reescribir las reglas de los pedidos.

Reflexión breve: tres errores encontrados, como los solucionaste y que aprendiste

Ruben Antonio Herrera Rodriguez:

Error 1 Decimal sin importar: Al ejecutar specifications.py salía un error porque Decimal no estaba importado. Lo solucioné agregando from decimal import Decimal al inicio del archivo. Aprendí que los valores monetarios se manejan con Decimal (no float) para evitar errores de redondeo, y que hay que importarlo explícitamente.

Error 2 Mensaje de rechazo mejorado en confirm(): El método rechazaba la confirmación con un solo mensaje que no se distinguía si el motivo era un cliente inactivo o un pedido sin artículos, y eso chocaba con el escenario E03 (O003, cliente inactivo). Lo solucioné separando las validaciones en domain.py: primero revisa si el cliente está activo ("El cliente esta inactivo") y luego si hay artículos ("El pedido requiere articulos"). Aprendí que los mensajes de error deben indicar la causa exacta, porque ayudan a depurar y a que la bitácora de escenarios sea clara.

Error 3: ModuleNotFoundError: No module named app. Al ejecutar el programa apareció este error porque lo corrí desde una carpeta equivocada. Lo solucioné ejecutando los comandos desde la carpeta raíz, donde está cargar_csv.py. Aprendí que importa desde dónde se ejecuta cada comando.

Blanca Jocelin Vázquez Reyna:

Tres errores que encontramos en el equipo y cómo los resolvimos:

1. Confundir los archivos y revolver el código: Al principio, por las prisas con el punto 12, terminé pegando el código de las pruebas dentro del archivo del proyecto (app/domain.py) y por eso todo tronaba con errores de importación y módulos no encontrados (ModuleNotFoundError). **¿Cómo lo arreglé?:** Usé un comando de Git (git restore) para dejar el archivo original limpio como estaba, y ya pasé las pruebas a su carpeta correspondiente (tests/test_domain.py), asegurándome de correr siempre los comandos desde la carpeta raíz del proyecto.

2. Querer cambiar el código original en lugar de ajustar la prueba (Atributos VIP): Las pruebas de los descuentos fallaban porque el sistema usaba is_vip y yo quería modificar a fuerzas la clase Order para que aceptara otra cosa, lo cual chocaba con las reglas del dominio. **¿Cómo lo arreglé?:** Me acordé de la indicación de no tocar el archivo domain.py. Así que mejor corregí la prueba usando el atributo real (is_vip) e importé una función que ya venía en el proyecto (discount_rate desde app.specifications) para validar las promos sin romper nada.

3. Que las pruebas de la página (API) se estorbaran entre sí: Las pruebas de los endpoints web tronaban porque todas intentaban usar los mismos números de pedido al mismo tiempo y se hacían bolas con la memoria. **¿Cómo lo arreglé?:** Separé las pruebas en test_api.py para que cada una fuera totalmente independiente. Le asigné un pedido diferente del archivo CSV a cada caso (por ejemplo, el O013 para la doble confirmación y el O012 para el envío) y ordené el código paso a paso: armar los datos, hacer la acción y revisar que saliera bien.

¿Qué me dejó de aprendizaje? Aprendí la regla de oro en el desarrollo: si una prueba falla, primero hay que revisar que la prueba esté bien hecha antes de querer moverle al código fuente que ya funciona. También entendí la importancia de estructurar las pruebas paso a paso usando el patrón de Preparar-Ejecutar-Comprobar (Arrange-Act-Assert), a cuidar desde qué carpeta ejecutamos los comandos en la terminal, y a ver que en las pruebas web es súper necesario que cada una vaya por su lado para no afectar los resultados de los demás.

Aaron Sebastian Torres Cabriales

1. **Falta de importaciones en las especificaciones:** Al momento de correr el archivo specifications.py, el programa se detuvo por completo porque faltaba importar el tipo de dato Decimal. **¿Cómo lo arreglé?:** Agregué la línea from decimal import Decimal al principio del archivo. Aprendí que en proyectos financieros o de pedidos siempre hay que usar Decimal en lugar de float para evitar errores raros en los centavos y los cálculos de dinero.
2. **Mensajes de rechazo confusos en el dominio:** El método para confirmar pedidos rechazaba las operaciones con un mensaje muy general, lo que causaba que chocara con las pruebas del escenario E03 cuando un cliente estaba inactivo. **¿Cómo lo arreglé?:** Modifiqué el código en domain.py para separar las validaciones y dar mensajes más claros: primero revisa si el cliente está inactivo y luego si el pedido no tiene artículos. Aprendí que ser específicos con los mensajes de error nos ahorra dolores de cabeza al momento de depurar.
3. **Problemas al ejecutar por la ubicación en la terminal:** Al intentar correr las pruebas, la consola me marcaba un error de que no encontraba el módulo app (ModuleNotFoundError). **¿Cómo lo arreglé?:** Me di cuenta de que estaba parado en una carpeta equivocada, así que me cambié a la carpeta raíz del proyecto (donde está el archivo cargar_csv.py) y volví a lanzar el comando. Aprendí que la ruta desde donde ejecutas la terminal es clave para que los scripts encuentren los archivos.

¿Qué me dejó de aprendizaje? Aprendí que cuidar los detalles pequeños, como importar bien los tipos de datos numéricos y estructurar los mensajes de error del sistema, hace que todo fluya mejor

Códigos

Aplicación:

app/specifications.py

```
#Corrije el error que daba con Decimal faltaba importarlo
from decimal import Decimal
#specification pattern
class Specification:
    def is_satisfied_by(self, order):
        raise NotImplementedError

    def __and__(self, other):
        return AndSpec(self, other)
    def __or__(self, other):
        return OrSpec(self, other)
    def __invert__(self):
        return NotSpec(self)

class AndSpec(Specification):
    def __init__(self, left, right):
        self.left, self.right = left, right
    def is_satisfied_by(self, order):
        return (self.left.is_satisfied_by(order)
                and self.right.is_satisfied_by(order))

class OrSpec(Specification):
    def __init__(self, left, right):
        self.left, self.right = left, right
    def is_satisfied_by(self, order):
```

```
        return (self.left.is_satisfied_by(order)
                or self.right.is_satisfied_by(order))

class NotSpec(Specification):
    def __init__(self, spec):
        self.spec = spec
    def is_satisfied_by(self, order):
        return (not self.spec.is_satisfied_by(order))

class MinimumAmountSpec(Specification):
    def __init__(self, minimum):
        self.minimum = minimum

    def is_satisfied_by(self, order):
        return order.total.amount > self.minimum


class VIPCustomerSpec(Specification):
    def is_satisfied_by(self, order):
        return order.customer.is_vip


class MinimumUnitsSpec(Specification):
    def __init__(self, minimum):
        self.minimum = minimum

    def is_satisfied_by(self, order):
        return order.units >= self.minimum
```

```
class ActivePromotionSpec(Specification):
    def is_satisfied_by(self, order):
        return order.has_promotion
# Regla de descuento de 10 %:
ten_percent = (
    MinimumAmountSpec(Decimal("500"))
    & VIPCustomerSpec()
    & MinimumUnitsSpec(4)
    & ~ActivePromotionSpec()
)
# Regla de 5 %: total > 5000 y SIN promoción.
five_percent = (
    MinimumAmountSpec(Decimal("5000"))
    & ~ActivePromotionSpec()
)
# Combina ambas reglas con OR.
# Si ambas se cumplen, aplica primero 10 %.
discount_eligible = ten_percent | five_percent
def discount_rate(order):
    if ten_percent.is_satisfied_by(order):
        return Decimal("0.10")

    if five_percent.is_satisfied_by(order):
        return Decimal("0.05")

    return Decimal("0.00")
```

app/domain.py

```
#money y cabecera
from dataclasses import dataclass, field
from decimal import Decimal
from enum import Enum
from datetime import datetime, timezone

class DomainException(Exception): pass
class InvalidOrderData(DomainException): pass
class InvalidOrderTransition(DomainException): pass
class OrderItemLimitExceeded(DomainException): pass

@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: str = "MXN"

    def __post_init__(self):
        if not self.amount.is_finite() or self.amount < 0:
            raise InvalidOrderData("Monto inválido")

    def add(self, other):
        if self.currency != other.currency:
            raise InvalidOrderData("Monedas distintas")
        return Money(self.amount + other.amount, self.currency)
#Customer y order item
@dataclass(frozen=True)
```

```
class Customer:
    id: str
    is_vip: bool = False
    active: bool = True

@dataclass(frozen=True)
class OrderItem:
    sku: str
    quantity: int
    unit_price: Money

    def __post_init__(self):
        if not self.sku.strip() or self.quantity <= 0:
            raise InvalidOrderData("Artículo inválido")

    @property
    def subtotal(self):
        return Money(self.unit_price.amount * self.quantity)
#order como agregate root
class OrderStatus(str, Enum):
    DRAFT = "DRAFT"
    CONFIRMED = "CONFIRMED"
    SHIPPED = "SHIPPED"
    DELIVERED = "DELIVERED"
    CANCELLED = "CANCELLED"
    RETURNED = "RETURNED"

@dataclass(frozen=True)
```

```
class DomainEvent:
    name: str
    order_id: str
    occurred_at: datetime = field(
        default_factory=lambda: datetime.now(timezone.utc)
    )

@dataclass
class Order:
    customer: Customer
    id: str
    has_promotion: bool = False
    status: OrderStatus = OrderStatus.DRAFT
    items: list[OrderItem] = field(default_factory=list)
    events: list = field(default_factory=list)

    @property
    def units(self):
        return sum(item.quantity for item in self.items)

    @property
    def total(self):
        amount = sum((x.subtotal.amount for x in self.items),
                     Decimal("0"))
        return Money(amount)

    def add_item(self, item):
        if self.status != OrderStatus.DRAFT:
```

```
            raise InvalidOrderTransition("Solo DRAFT admite artículos")
        if self.units + item.quantity > 20:
            raise OrderItemLimitExceeded("Máximo 20 unidades")
        if item.unit_price.currency != "MXN":
            raise InvalidOrderData("Solo MXN")
        self.items.append(item)

    def confirm(self):
        if self.status != OrderStatus.DRAFT:
            raise InvalidOrderTransition("Solo DRAFT puede confirmarse")

        #Hice el cambio para que muestre mas especificamente la causa del rechazo ya que choca con el O003
        if not self.customer.active:
            raise InvalidOrderData("El cliente esta inactivo")

        if not self.items:
            raise InvalidOrderData("El pedido requiere articulos")

        self.status = OrderStatus.CONFIRMED
        self.events.append(DomainEvent("OrderConfirmed", self.id))

    def ship(self, tracking_number):
        if self.status != OrderStatus.CONFIRMED:
            raise InvalidOrderTransition(
                "Solo CONFIRMED puede enviarse"
            )
```

```
    if not tracking_number or not tracking_number.strip():
        raise InvalidOrderData("Se requiere una guía de envío")

    self.status = OrderStatus.SHIPPED
    self.events.append(DomainEvent("OrderShipped", self.id))

def cancel(self, reason):
    if self.status != OrderStatus.CONFIRMED:
        raise InvalidOrderTransition(
            "Solo CONFIRMED puede cancelarse"
        )

    if not reason or not reason.strip():
        raise InvalidOrderData("Se requiere un motivo de cancelación")

    self.status = OrderStatus.CANCELLED
    self.events.append(DomainEvent("OrderCancelled", self.id))

def deliver(self):
    if self.status != OrderStatus.SHIPPED:
        raise InvalidOrderTransition(
            "Solo SHIPPED puede marcarse como entregado"
        )

    self.status = OrderStatus.DELIVERED
    self.events.append(DomainEvent("OrderDelivered", self.id))

def return_order(self):
```

```
        if self.status != OrderStatus.SHIPPED:
            raise InvalidOrderTransition(
                "Solo SHIPPED puede devolverse"
            )

        self.status = OrderStatus.RETURNED
        self.events.append(DomainEvent("OrderReturned", self.id))
```

app/api.py

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from cargar_csv import load_fixture
from app.domain import (
    InvalidOrderData,
    InvalidOrderTransition,
    OrderItemLimitExceeded,
)

app = FastAPI(title="TechStore U4")

#Los fixtures se cargan al iniciar y viven en memoria.
orders = load_fixture()

class ShipRequest(BaseModel):
    tracking_number: str


class CancelRequest(BaseModel):
```

```
    reason: str


#Funciones de apoyo
def get_order_or_404(order_id: str):
    order = orders.get(order_id)
    if order is None:
        raise HTTPException(status_code=404, detail="No existe")
    return order


def run_action(order, action):
    #Ejecuta un metodo de Order y traduce las excepciones de dominio a HTTP.
    try:
        action()
    except InvalidOrderTransition as exc:
        raise HTTPException(status_code=409, detail=str(exc))
    except OrderItemLimitExceeded as exc:
        raise HTTPException(status_code=409, detail=str(exc))
    except InvalidOrderData as exc:
        raise HTTPException(status_code=422, detail=str(exc))
    return {"id": order.id, "status": order.status.value}


#Endpoints
@app.get("/orders/{order_id}")
def get_order(order_id: str):
    order = get_order_or_404(order_id)
```

```
    return {
        "id": order.id,
        "customer_id": order.customer.id,
        "status": order.status.value,
        "has_promotion": order.has_promotion,
        "units": order.units,
        "total": str(order.total.amount),
        "currency": order.total.currency,
        "items": [
            {
                "sku": i.sku,
                "quantity": i.quantity,
                "unit_price": str(i.unit_price.amount),
                "subtotal": str(i.subtotal.amount),
            }
            for i in order.items
        ],
        "events": [e.name for e in order.events],
    }

@app.post("/orders/{order_id}/confirm")
def confirm_order(order_id: str):
    order = get_order_or_404(order_id)
    return run_action(order, order.confirm)

@app.post("/orders/{order_id}/ship")
def ship_order(order_id: str, body: ShipRequest):
    order = get_order_or_404(order_id)
```

```
    return run_action(order, lambda: order.ship(body.tracking_number))

@app.post("/orders/{order_id}/cancel")
def cancel_order(order_id: str, body: CancelRequest):
    order = get_order_or_404(order_id)
    return run_action(order, lambda: order.cancel(body.reason))
```

Código de pruebas

tests/test_api.py

```
from fastapi.testclient import TestClient
from app.api import app

client = TestClient(app)

def test_obtener_pedido_404():
    # Ejecutar y comprobar
    respuesta = client.get("/orders/X999")
    assert respuesta.status_code == 404

def test_confirmar_pedido_conflicto_409():
    # Preparar: confirmamos la primera vez
    client.post("/orders/O013/confirm")

    # Ejecutar: confirmamos la segunda vez
    respuesta = client.post("/orders/O013/confirm")

    # Comprobar
```

```
    assert respuesta.status_code == 409


def test_envio_sin_guia():
    # Preparar: confirmamos el pedido O012 primero
    client.post("/orders/O012/confirm")

    # Ejecutar: enviamos sin guía
    respuesta = client.post("/orders/O012/ship", json={"tracking_number": " "})

    # Comprobar
    assert respuesta.status_code == 422
```

tests/test_domain.py

```
import pytest
from decimal import Decimal
from cargar_csv import load_fixture
from app.domain import Order, Customer, OrderItem, Money, OrderItemLimitExceeded, InvalidOrderData, InvalidOrderTransition
from app.specifications import discount_rate


def test_money_negativo():
    with pytest.raises(InvalidOrderData):
        Money(Decimal("-1"))


def test_pedido_o001_inicia_draft():
    orders = load_fixture()
    assert orders["O001"].status.value == "DRAFT"
    assert orders["O001"].units == 4
```

```
    assert orders["O001"].total.amount == Decimal("2100.00")


def test_limite_20_unidades_por_pedido():
    cliente = Customer(id="C01", active=True)
    pedido = Order(id="0001", customer=cliente)
    pedido.add_item(OrderItem(sku="SKU1",                    quantity=15,
unit_price=Money(Decimal("100.00"))))
    with pytest.raises(OrderItemLimitExceeded):
        pedido.add_item(OrderItem(sku="SKU2",                    quantity=6,
unit_price=Money(Decimal("50.00"))))


def test_cliente_inactivo():
    cliente = Customer(id="C02", active=False)
    pedido = Order(id="0002", customer=cliente)
    with pytest.raises(InvalidOrderData):
        pedido.confirm()


def test_pedido_vacio_no_se_confirma():
    cliente = Customer(id="C03", active=True)
    pedido = Order(id="0003", customer=cliente)
    with pytest.raises(InvalidOrderData):
        pedido.confirm()


def test_descuento_vip():
    # Preparar datos: is_vip=True, sin promoción, 4 unidades, total > 500
    cliente = Customer(id="C_VIP", is_vip=True, active=True)
    pedido = Order(id="0004", customer=cliente, has_promotion=False)
    pedido.add_item(OrderItem(sku="SKU_VIP",                    quantity=4,
unit_price=Money(Decimal("150.00"))))
```

```
    # Ejecutar y comprobar
    assert discount_rate(pedido) == Decimal("0.10")


def test_promocion_activa():
    # Preparar datos: is_vip=True, CON promoción, 4 unidades
    cliente = Customer(id="C_PROM", is_vip=True, active=True)
    pedido = Order(id="0005", customer=cliente, has_promotion=True)
    pedido.add_item(OrderItem(sku="SKU_PROM",                quantity=4,
unit_price=Money(Decimal("150.00"))))


    # Ejecutar y comprobar
    assert discount_rate(pedido) == Decimal("0.00")


def test_cancelacion_desde_shipped():
    cliente = Customer(id="C05", active=True)
    pedido = Order(id="0006", customer=cliente)
    pedido.add_item(OrderItem(sku="SKU1",                quantity=1,
unit_price=Money(Decimal("100.00"))))
    pedido.confirm()
    pedido.ship("TRACK123")
    with pytest.raises(InvalidOrderTransition):
        pedido.cancel("Ya no lo quiero")
```

.gitignore

```
.venv/
__pycache__/
```

.pytest_cache/

README.txt

TechStore U4

Estructura del proyecto

```
techstore-u4/
  datos/              CSV de clientes, productos, pedidos, detalle y escenarios
  cargar_csv.py       Carga los CSV y crea los objetos del dominio
  app/
    __init__.py
    domain.py         Money, Customer, OrderItem, Order, DomainEvent y excepciones
    specifications.py Specification Pattern y reglas de descuento
    api.py            API REST con FastAPI
  tests/
    test_domain.py    Pruebas del dominio
    test_api.py       Pruebas de la API
  probar_pt8.py Verificaba que el punto estaba bien hecho mediante pruebas
  probar_pt9.py Verificaba que el punto estaba bien hecho mediante pruebas
  probar_pt10.py Verificaba que el punto estaba bien hecho mediante pruebas
  Bitacora_15escenarios.csv Bitacora creada desde probar_pt10.py para completar el punto 10
  evidencias/ Contiene imagenes de evidencias
    Carga_16pedidos.png
    Swagger_404.png
    Swagger_exito.png
```

Swagger_422.png

Swagger_409.png

```

## Instalación

Requiere Python 3.11 o superior.

1. Clonar el repositorio y entrar a la carpeta raíz del proyecto (la que contiene `cargar_csv.py`).

2. Crear y activar un entorno virtual:

```
python -m venv .venv

Windows PowerShell: .venv\Scripts\Activate.ps1

Windows CMD: .venv\Scripts\activate.bat

macOS / Linux: source .venv/bin/activate
```

3. Instalar las dependencias:

```
python -m pip install fastapi uvicorn pytest httpx
```

Ejecución

Todos los comandos se ejecutan desde la carpeta raíz del proyecto.
```

Cargar los datos (debe mostrar los 16 pedidos en estado DRAFT):

```
python cargar_csv.py
```

Levantar la API:

```
uvicorn app.api:app --reload
```

- Documentación interactiva (Swagger): http://127.0.0.1:8000/docs
- Contrato OpenAPI: http://127.0.0.1:8000/openapi.json

Los pedidos se cargan al iniciar el servidor y viven en memoria. Para repetir una prueba desde cero, reinicia el servidor.

Ejecutar las pruebas:

```
python -m pytest -q
```

Reglas de negocio

Pedido (Order)

- Solo un pedido en DRAFT admite artículos.
- El límite es de 20 unidades por pedido (suma de cantidades).
- Solo se admite la moneda MXN.
- Los montos usan `Decimal`; no pueden ser negativos.
- Cada artículo requiere un SKU no vacío y una cantidad mayor a cero.

Descuentos (implementados con Specifications combinadas con `&`, `|` y `~`)

- **10 %:** cliente VIP, total mayor a 500, al menos 4 unidades y sin promoción activa.
- **5 %:** total mayor a 5000 y sin promoción activa.
- Si se cumplen ambas reglas, se aplica primero el 10 %.
- Una promoción activa bloquea ambos descuentos.
- El umbral de 500 es estricto: un total exactamente igual a 500 no cumple.

Estados y Domain Events

```
DRAFT --confirm--> CONFIRMED --ship--> SHIPPED --deliver--> DELIVERED
                     |                |
                  cancel            return
                     |                |
                CANCELLED          RETURNED
```

Operación	Estado requerido	Validación adicional	Evento
`confirm()`	DRAFT	Cliente activo y pedido con artículos	`OrderConfirmed`

| `ship(tracking_number)` | CONFIRMED | Guía de envío no vacía | `OrderShipped` |

| `cancel(reason)` | CONFIRMED | Motivo no vacío | `OrderCancelled` |

| `deliver()` | SHIPPED | – | `OrderDelivered` |

| `return_order()` | SHIPPED | – | `OrderReturned` |

En todos los métodos se valida primero, después se cambia el estado y al final se registra el evento. Si una operación es inválida, el pedido no cambia y no se agrega ningún evento.

Endpoints

| Método | Ruta | Cuerpo | Descripción |

|---|---|---|---|

| GET | `/orders/{id}` | – | Consulta un pedido (estado, unidades, total, artículos y eventos) |

| POST | `/orders/{id}/confirm` | – | Confirma un pedido |

| POST | `/orders/{id}/ship` | `{"tracking_number": "G123"}` | Envía un pedido confirmado |

| POST | `/orders/{id}/cancel` | `{"reason": "Motivo"}` | Cancela un pedido confirmado |

Códigos de respuesta

| Código | Cuándo ocurre | Ejemplo |

|---|---|---|

| 200 | La operación fue exitosa | Confirmar O013 por primera vez |

| 404 | El pedido no existe | Confirmar `X999` |

| 409 | Transición de estado no permitida | Confirmar O013 una segunda vez |

| 422 | Datos inválidos | Confirmar O007 (pedido sin artículos) |

cargar_csv.py

```
"""Carga fixtures TechStore desde CSV en objetos del dominio de U4.
Ejecutar desde la carpeta que contiene 'datos' y este archivo.
"""
import csv
from pathlib import Path
from decimal import Decimal
from app.domain import Customer, Order, OrderItem, Money

BASE = Path(__file__).resolve().parent / "datos"

def rows(filename):
    with (BASE / filename).open(encoding="utf-8-sig", newline="") as f:
        return list(csv.DictReader(f))

def load_fixture():
    clients = {
        x["cliente_id"]: Customer(
            x["cliente_id"], bool(int(x["es_vip"])), bool(int(x["activo"]))
        ) for x in rows("clientes.csv")
    }
    prices = {x["sku"]: Decimal(x["precio_mxn"]) for x in rows("productos.csv")}
    orders = {
        x["pedido_id"]: Order(
            customer=clients[x["cliente_id"]],
            id=x["pedido_id"],
            has_promotion=bool(int(x["tiene_promocion"]))
```

```
        ) for x in rows("pedidos.csv")
    }
    for x in rows("detalle_pedidos.csv"):
        orders[x["pedido_id"]].add_item(
            OrderItem(x["sku"], int(x["cantidad"]), Money(prices[x["sku"]]))
        )
    return orders

if __name__ == "__main__":
    orders = load_fixture()
    for order_id, order in orders.items():
        print(order_id, order.status.value, order.units, order.total.amount)
```

probar_pt10.py

```
import csv
from decimal import Decimal

from cargar_csv import load_fixture, rows
from app.domain import (
    Money,
    OrderItem,
    InvalidOrderData,
    InvalidOrderTransition,
    OrderItemLimitExceeded,
)
from app.specifications import discount_rate


resultados = []


def estado(order):
    return order.status.value


def registrar(caso, inicial, operacion, resultado, explicacion):
    resultados.append({
        "caso_id": caso,
        "estado_inicial": inicial,
        "operacion": operacion,
        "resultado_real": resultado,
```

```
        "explicacion": explicacion,
    })


def ejecutar(order, descripcion, funcion):
    #Ejecuta una operación y registra su resultado y sus eventos.
    eventos_antes = len(order.events)

    try:
        funcion()
        resultado = f"Éxito; estado actual: {estado(order)}"
    except (
        InvalidOrderData,
        InvalidOrderTransition,
        OrderItemLimitExceeded,
    ) as error:
        resultado = f"Rechazo: {error}"

    eventos_nuevos = [
        evento.name for evento in order.events[eventos_antes:]
    ]

    if eventos_nuevos:
        eventos = ", ".join(eventos_nuevos)
    else:
        eventos = "ninguno"

    return f"{descripcion}: {resultado}. Eventos nuevos: {eventos}."
```

```
def nuevo_pedido(pedido_id):
    # Cada escenario comienza con los datos originales.
    return load_fixture()[pedido_id]


def registrar_descuento(caso, pedido_id, esperado, explicacion):
    order = nuevo_pedido(pedido_id)
    inicial = estado(order)
    descuento = discount_rate(order)
    real = f"{descuento * 100}%"

    registrar(
        caso,
        inicial,
        "Evaluar descuento",
        real,
        f"{explicacion}. Esperado: {esperado}. "
        "La evaluación no cambia el pedido ni genera eventos.",
    )


# E01: descuento del 10 %
registrar_descuento(
    "E01", "O001", "10%",
    "Regla VIP, monto mayor a 500, al menos 4 unidades y sin promoción"
)
```

```
# E02: sin descuento
registrar_descuento(
    "E02", "O002", "0%",
    "Cliente no VIP y total menor a 5000, según el escenario"
)

# E03: cliente inactivo
order = nuevo_pedido("O003")
inicial = estado(order)
detalle = ejecutar(order, "Confirmar", order.confirm)
registrar(
    "E03", inicial, "Confirmar", detalle,
    "Se espera rechazo por cliente inactivo. "
    f"Estado final: {estado(order)}."
)

# E04: la promocion bloquea el descuento
registrar_descuento(
    "E04", "O005", "0%",
    "La promocion activa impide aplicar ambos descuentos"
)

# E05: descuento del 5 %
registrar_descuento(
    "E05", "O006", "5%",
    "Total mayor a 5000 y sin promocion"
)
```

```
# E06: pedido sin articulos
order = nuevo_pedido("O007")
inicial = estado(order)
detalle = ejecutar(order, "Confirmar", order.confirm)
registrar(
    "E06", inicial, "Confirmar", detalle,
    "Se espera rechazo porque el pedido no tiene articulos. "
    f"Estado final: {estado(order)}."
)

# E07: pedido con 20 unidades
order = nuevo_pedido("O008")
inicial = estado(order)
unidades = order.units
detalle = ejecutar(order, "Confirmar", order.confirm)
registrar(
    "E07", inicial, "Confirmar", detalle,
    f"Unidades iniciales: {unidades}. El limite permitido es 20. "
    f"Estado final: {estado(order)}."
)

# E08: intentar agregar 2 unidades y exceder el limite
order = nuevo_pedido("O009")
inicial = estado(order)

productos = {
    fila["sku"]: Decimal(fila["precio_mxn"])
```

```
    for fila in rows("productos.csv")
}
precio = productos["SKU003"]

detalle = ejecutar(
    order,
    "Agregar SKU003 x2",
    lambda: order.add_item(
        OrderItem("SKU003", 2, Money(precio))
    ),
)

registrar(
    "E08", inicial, "Agregar SKU003 x2", detalle,
    f"Unidades antes: {order.units}. Se intenta agregar 2 unidades. "
    f"Unidades despues: {order.units}. Se espera rechazo si se exceden 20."
)

# E09: el umbral es estricto: total > 500
registrar_descuento(
    "E09", "O010", "0%",
    "Un total exactamente igual a 500 no supera el umbral"
)

# E10: solo 3 unidades
registrar_descuento(
    "E10", "O011", "0%",
    "La regla del 10% exige al menos 4 unidades"
```

```
)

# E11: confirmar si, enviar sin guia no
order = nuevo_pedido("O012")
inicial = estado(order)
pasos = []

pasos.append(ejecutar(order, "Confirmar", order.confirm))
pasos.append(
    ejecutar(order, "Enviar sin guia", lambda: order.ship(""))
)

registrar(
    "E11", inicial, "Confirmar y enviar sin guia",
    " | ".join(pasos),
    f"La confirmación debe ser valida y el envio rechazado por falta de guia. "
    f"Estado final: {estado(order)}."
)

# E12: confirmar dos veces
order = nuevo_pedido("O013")
inicial = estado(order)
pasos = []

pasos.append(ejecutar(order, "Primera confirmacion", order.confirm))
pasos.append(ejecutar(order, "Segunda confirmacion", order.confirm))

registrar(
```

```
    "E12", inicial, "Confirmar dos veces",
    " | ".join(pasos),
    f"La segunda confirmacion debe rechazarse. Estado final: {estado(order)}."
)

# E13: cancelar despues de enviar
order = nuevo_pedido("O014")
inicial = estado(order)
pasos = []

pasos.append(ejecutar(order, "Confirmar", order.confirm))
pasos.append(
    ejecutar(order, "Enviar", lambda: order.ship("GUIA-001"))
)
pasos.append(
    ejecutar(order, "Cancelar", lambda: order.cancel("Cancelacion de prueba"))
)

registrar(
    "E13", inicial, "Confirmar, enviar y cancelar",
    " | ".join(pasos),
    f"La cancelacion debe rechazarse en SHIPPED. Estado final: {estado(order)}."
)

# E14: devolver pedido enviado
order = nuevo_pedido("O015")
inicial = estado(order)
pasos = []
```

```
pasos.append(ejecutar(order, "Confirmar", order.confirm))
pasos.append(
    ejecutar(order, "Enviar", lambda: order.ship("GUIA-002"))
)
pasos.append(ejecutar(order, "Devolver", order.return_order))

registrar(
    "E14", inicial, "Confirmar, enviar y devolver",
    " | ".join(pasos),
    f"Se espera terminar en RETURNED. Estado final: {estado(order)}."
)

# E15: entregar pedido enviado
order = nuevo_pedido("O016")
inicial = estado(order)
pasos = []

pasos.append(ejecutar(order, "Confirmar", order.confirm))
pasos.append(
    ejecutar(order, "Enviar", lambda: order.ship("GUIA-003"))
)
pasos.append(ejecutar(order, "Entregar", order.deliver))

registrar(
    "E15", inicial, "Confirmar, enviar y entregar",
    " | ".join(pasos),
    f"Se espera terminar en DELIVERED. Estado final: {estado(order)}."
```

```
)


# Mostrar los resultados en pantalla
for resultado in resultados:
    print("-" * 80)
    for columna, valor in resultado.items():
        print(f"{columna}: {valor}")


# Guardar el registro en CSV
with open(
    "Bitacora_15escenarios.csv",
    "w",
    newline="",
    encoding="utf-8-sig",
) as archivo:
    columnas = [
        "caso_id",
        "estado_inicial",
        "operacion",
        "resultado_real",
        "explicacion",
    ]
    escritor = csv.DictWriter(archivo, fieldnames=columnas)
    escritor.writeheader()
    escritor.writerows(resultados)
```

```
print("\nSe guardo todo en Bitacora_15escenarios.csv")
```

probar_pt8.py

```
from decimal import Decimal
from cargar_csv import load_fixture
from app.specifications import discount_rate

orders = load_fixture()

for order_id in ["O001", "O006", "O005", "O010"]:
    order = orders[order_id]
    rate = discount_rate(order)

    print(
        order_id,
        "Total:", order.total.amount,
        "Descuento:", rate * 100, "%"
    )
```

probar_pt9.py

```
from cargar_csv import load_fixture
from app.domain import InvalidOrderTransition

orders = load_fixture()
order = orders["O013"]

print("Estado inicial:", order.status)
print("Eventos iniciales:", len(order.events))
```

```
# Primera confirmacion
try:
    order.confirm()
    print("Primera confirmación:", order.status)
    print("Eventos:", [event.name for event in order.events])
except Exception as error:
    print("Error:", error)

# Segunda confirmacion
try:
    order.confirm()
except InvalidOrderTransition as error:
    print("Segunda confirmación rechazada:", error)

print("Estado final:", order.status)
print("Total de eventos:", len(order.events))
```